MICROPROCESSOR & MICROCONTROLLER

Complete Exam Answer Guide

Covering All Frequently Asked Exam Questions

Source: Course PDF Chapters 1-5 & Class Notes (Faria)

CHAPTER 1: MICROCOMPUTER SYSTEMS & 8086 ARCHITECTURE

Q1. What is an N-bit processor? What is a 16-bit processor?

An N-bit processor is a processor whose ALU (Arithmetic Logic Unit) can handle N bits of data at a time. The 'N' refers to the size of the data bus.

A 16-bit processor is one whose ALU, internal registers, and most instructions are designed to work with 16-bit binary words.

Examples:

- 8085 = 8-bit processor (data bus = 8-bit)
- 8086 = 16-bit processor (data bus = 16-bit)
- 8088 = 16-bit processor internally, but 8-bit external data bus (exception)

(Source: Chapter 1 - Microcomputer Systems PDF)

Q2. Draw and explain the block diagram of Intel 8086 Microprocessor (EU & BIU)

The 8086 has two main components:

(A) Execution Unit (EU):

The EU executes instructions. It contains:

- ALU - performs arithmetic (+, -, x, /) and logic (AND, OR, NOT) operations
- 8 General Purpose Registers: AX, BX, CX, DX, SI, DI, BP, SP
- Temporary Registers - hold operands for the ALU
- FLAGS Register - 16-bit register with 9 active flags indicating result conditions

(B) Bus Interface Unit (BIU):

The BIU provides communication between EU and memory/I/O port. It contains:

- CS (Code Segment) - holds segment number of code
- DS (Data Segment) - holds segment number of data
- SS (Stack Segment) - holds segment number of stack
- ES (Extra Segment) - holds extra segment number
- IP (Instruction Pointer) - holds offset of next instruction
- Instruction Queue - 6-byte queue (prefetches instructions)
- Bus Control Logic - manages address, data, and control signals

8086 Key Specifications:

- 16-bit data bus - can read/write 16-bits or 8-bits at a time
- 20-bit address bus - can address 2^20 = 1MB memory locations

(Source: Chapter 1 - PDF Fig 1.6)

Q3. Address capability of 8086 and I/O lines

Feature	8086 Specification
Address Bus Width	20-bit
Memory Address Capability	2^20 = 1,048,576 = 1 Megabyte (1 MB)
Data Bus Width	16-bit
I/O Ports Accessible	2^16 = 65,536 (64K) I/O ports
Data Read/Write	16-bit or 8-bit at a time

Q4. Compare/Differentiate 8085 and 8086 Microprocessor

Characteristic	8085	8086
Data Bus	8-bit	16-bit
Address Bus	16-bit	20-bit
Memory Access	64 KB (2^16)	1 MB (2^20)
Type	8-bit processor	16-bit processor
ALU	8-bit ALU	16-bit ALU
Registers	8-bit general registers	16-bit general registers
FLAGS	5 status flags	9 flags (6 status + 3 control)
Instruction Queue	No	6-byte queue

Characteristic	8085	8086
Memory Banking	Not used	Even/Odd address banking
Processing Speed	Slower	Faster (16x more memory access)

Memory access ratio: 2^20 / 2^16 = 2^4 = 16 times more than 8085

(Source: Class Notes + Chapter 1 PDF)

Q5. Discuss different categories of functions performed by a microprocessor

All microprocessor functions are classified into 3 categories:

1. Microprocessor-Initiated Operations (MPU performs 4 operations):

- Memory Read: Reads data FROM memory
- Memory Write: Writes data INTO memory
- I/O Read: Accepts data FROM input devices
- I/O Write: Sends data TO output devices

Steps to communicate with peripherals/memory:

1. Identify the peripheral or memory location (with its address)
2. Transfer data
3. Provide timing or synchronization signals

2. Internal Data Operations (8085/8086):

- Store 8-bit or 16-bit data in registers
- Perform arithmetic and logical operations
- Test for conditions (using flags)
- Sequence the execution of instructions
- Store data temporarily in the stack

3. Peripheral (Externally) Initiated Operations:

- Reset: Clears Program Counter (PC) to 0000H, suspends all operations
- Interrupt: Processor stops normal execution, runs ISR (Interrupt Service Routine), then resumes
- Ready: If READY pin is LOW, processor enters Wait state (for slow devices)
- Hold: Processor gives up control of system bus for external devices (used in DMA)

(Source: Chapter 2 - Microprocessor Architecture PDF)

CHAPTER 2: BUS MULTIPLEXING & DEMULTIPLEXING

Q6. How are address and data buses of 8086 multiplexed and demultiplexed?

Why Multiplexing?

The 8086 needs 20 address lines + 16 data lines = 36 lines. To reduce pin count, the lower 16 address lines (A0-A15) are shared (multiplexed) with the 16 data lines (D0-D15). These are called AD0-AD15 (Address/Data lines).

Multiplexed Bus Structure:

- Pins AD0-AD15: Multiplexed address (lower 16 bits) and data lines
- Pins A16-A19 / S3-S6: Higher order address lines (also multiplexed with status)
- Total Pins: 20 address + 16 data = sharing 20 pins

How Multiplexing Works (Two Phases):

4. Address Phase (T1 cycle): ALE = HIGH, pins carry address information (A0-A19)
5. Data Phase (T2-T4 cycles): ALE = LOW, same pins carry data (D0-D15)

Demultiplexing Process:

ALE (Address Latch Enable) pin is used to demultiplex:

- When ALE = 1 (HIGH): The pins carry ADDRESS - latch the address into external latch (74LS373)
- When ALE = 0 (LOW): The pins carry DATA - processor reads/writes data
- External latch (74LS373/8282) holds the address while data transfer occurs

(Source: Class Notes - 23.11.25 + Chapter 1 PDF)

Q7. Explain the function of important pins of 8086

Pin Name	Full Name	Function
MN/MX	Minimum/Maximum Mode	HIGH = Minimum mode (single processor). LOW = Maximum mode (multiprocessor system)
READY	Ready Signal	If LOW, processor enters Wait state to synchronize with slow peripherals
ALE	Address Latch Enable	HIGH during T1 = address on bus. LOW = data on bus. Used for demultiplexing
DT/R	Data Transmit/Receive	HIGH = data is being transmitted (write). LOW = data is being received (read). Used with bus transceiver
RD	Read	LOW = processor is reading from memory or I/O port
WR	Write	LOW = processor is writing to memory or I/O port
RESET	Reset	Clears registers, PC goes to 0000H

CHAPTER 3: MEMORY SEGMENTATION & ADDRESS GENERATION

Q8. What is memory segmentation of 8086? Explain the concept.

Memory segmentation is a technique used by the 8086 to access its 1 MB (20-bit) address space using 16-bit registers.

Why Segmentation?

- 8086 registers are 16-bit, which can address only 64 KB (2^16)
- 8086 needs to address 1 MB (2^20) total memory
- Solution: Divide memory into segments of 64 KB each

Memory Segment Definition:

A memory segment is a block of 2^16 (64K = 65,536) consecutive memory bytes.

- Each segment is identified by a 16-bit segment number
- Segment numbers range from 0000H to FFFFH
- Segments start at paragraph boundaries (every 16 bytes / 10H)
- Segments can overlap each other

Four Active Segments at any time:

- Code Segment (CS) - stores program instructions
- Data Segment (DS) - stores program data/variables
- Stack Segment (SS) - stores the stack
- Extra Segment (ES) - additional data storage

(Source: Chapter 3 - PDF)

Q9. Discuss different segment registers and offset registers of 8086

Segment Registers (in BIU):

Register	Full Name	Role / Used with
CS	Code Segment	Holds start address of code. Used with IP (CS:IP = next instruction)
DS	Data Segment	Holds start address of data. Used with SI (DS:SI = data access)
SS	Stack Segment	Holds start address of stack. Used with SP and BP (SS:SP, SS:BP)
ES	Extra Segment	Used for extra data, especially string operations. Used with DI (ES:DI)

Offset / Pointer / Index Registers (in EU):

Register	Full Name	Function
SP	Stack Pointer	Points to top of stack. Used with SS
BP	Base Pointer	Used to access data on stack or other segments
SI	Source Index	Points to source data in DS. Incremented for consecutive access
DI	Destination Index	Points to destination in ES. Used in string operations
IP	Instruction Pointer	Contains offset of next instruction to execute (used with CS)

(Source: Chapter 3 - PDF Sections 3.2.3, 3.2.4, 3.2.5)

Q10. Physical Address Generation in 8086 - Steps and Examples

Formula:

Physical Address = Segment Register x 10H + Offset

(Multiplying by 10H = shifting 4 bits to the left)

Steps:

6. Take the 16-bit segment number from segment register
7. Shift it 4 bits left (multiply by 10H) to get a 20-bit value
8. Add the 16-bit offset value
9. Result = 20-bit physical address

Example 1: Segment:Offset = A4FB:4872

```
  A4FB0H    (segment A4FBH shifted left 4 bits)
+ 4872H     (offset)
-----------
  A9822H    (Physical Address)
```

Example 2 (from exam): Physical address 1256AH, find offset in segment 1256H

```
Physical Address = 1256AH
Segment = 1256H => 12560H (after shift)
Offset X = 1256AH - 12560H = AH = 000AH
So: 1256AH = 1256:000AH
```

Example 3: Physical address 1256AH in segment 1240H

```
Segment = 1240H => 12400H (after shift)
Offset Y = 1256AH - 12400H = 16AH
So: 1256AH = 1240:016AH
```

Example 4 (from exam): Physical 80FBDH, offset BFD2H - find segment

```
Segment x 10H = Physical Address - Offset
Physical = 80FD2H
- Offset  = BFD2H
-------------------
Segment x 10H = 75000H
```

```
Segment = 7500H
```

(Source: Chapter 3 - Examples 3.1, 3.2 PDF)

CHAPTER 4: MEMORY BANKING & ODD/EVEN ADDRESSING

Q11. What is Memory Banking? Explain Odd and Even Addressing in 8086

Memory Banking Concept:

The 8086 divides its 1 MB memory into two 512 KB banks:

- Even Address Bank (Low Bank): Contains memory at addresses 00000H, 00002H, 00004H...FFFFEH
- Odd Address Bank (High Bank): Contains memory at addresses 00001H, 00003H, 00005H...FFFFFH

Why Two Banks?

- 8086 has a 16-bit data bus but can also do 8-bit operations
- Even bank connected to D0-D7 (lower byte), Odd bank connected to D8-D15 (upper byte)

Reading Data:

Access Type	Location	Machine Cycles	Notes
Byte (8-bit)	Even Boundary	M1 (1 cycle)	Fast - no penalty
Byte (8-bit)	Odd Boundary	M1 (1 cycle)	Slightly different bus usage
Word (16-bit)	Even Boundary	M1 (1 cycle)	Both bytes in one cycle - FAST
Word (16-bit)	Odd Boundary	M1 + M2 (2 cycles)	SLOW - needs extra machine cycle

Key Rules:

- Even Boundary: Address divisible by 2 (ends in 0, 2, 4, 6, 8)
- Odd Boundary: Address not divisible by 2 (ends in 1, 3, 5, 7, 9)
- Paragraph Boundary: Address divisible by 16 (ends in 0H)
- Word at even boundary = 1 machine cycle (efficient)
- Word at odd boundary = 2 machine cycles (penalty)

(Source: Class Notes 23.11.25 + Chapter 3 PDF)

CHAPTER 5: FLAGS REGISTER

Q12. What are Flags? Discuss FLAGS Register of 8086

A flag is a flip-flop that indicates some condition produced by the execution of an instruction. The 8086 has a 16-bit FLAGS register with 9 active flags.

Layout of FLAGS Register:

Bit	15-12	11 (OF)	10 (DF)	9 (IF)	8 (TF)	7 (SF)	6 (ZF)	5	4 (AF)	3	2 (PF)	1	0 (CF)
Use	-	Status	Control	Control	Control	Status	Status	-	Status	-	Status	-	Status

6 Status Flags:

Flag	Name	Set (=1) when...
CF	Carry Flag	Carry out from MSB on addition, or borrow into MSB on subtraction
PF	Parity Flag	Low byte of result has EVEN number of 1-bits (even parity)
AF	Auxiliary Carry Flag	Carry out from bit 3 (used in BCD operations)
ZF	Zero Flag	Result is ZERO
SF	Sign Flag	MSB of result is 1 (result is negative for signed interpretation)
OF	Overflow Flag	Signed overflow occurred (result too large/small for signed range)

3 Control Flags:

Flag	Name	Function
TF	Trap Flag	Used for single-step debugging (one instruction at a time)
IF	Interrupt Flag	Controls maskable interrupts (1=enabled, 0=disabled)
DF	Direction Flag	Controls string operation direction (0=increment, 1=decrement)

Effect of Instructions on Flags:

Instruction	Flags Affected
MOV / XCHG	NONE - no flags changed
ADD / SUB	ALL flags affected (CF, PF, AF, ZF, SF, OF)
INC / DEC	All flags EXCEPT CF
NEG	All flags (CF=1 unless result=0; OF=1 if operand is 8000H or 80H)

(Source: Chapter 5 - Flags Register PDF)

Q13. Determine Status Flags after ADD AX, BX (AX=FFFFH, BX=FFFFH)

Step 1: Perform the addition

```
  FFFFH  = 1111 1111 1111 1111
+ FFFFH  = 1111 1111 1111 1111
-----------------------------
```

```
1 FFFEH  = 1 1111 1111 1111 1110

Stored in AX = FFFEH = 1111 1111 1111 1110
```

Step 2: Determine each flag

Flag	Value	Reason
CF (Carry)	1	There IS a carry out from the MSB (the leading 1 is lost)
ZF (Zero)	0	Result FFFEH is NOT zero
SF (Sign)	1	MSB of FFFEH is 1 (result appears negative in signed interpretation)
PF (Parity)	0	Low byte = FEH = 1111 1110 = seven 1-bits = ODD parity, so PF=0
OF (Overflow)	0	No signed overflow: FFFFH=-1 signed, -1+(-1)=-2=FFFEH, which is correct
AF (Auxiliary)	1	Carry from bit 3 to bit 4 exists in this addition

Final Answer:

- CF = 1 (unsigned overflow)
- ZF = 0 (result not zero)
- SF = 1 (MSB is 1)
- PF = 0 (odd parity in low byte)
- OF = 0 (no signed overflow)

(Source: Chapter 5 - Example 5.1 PDF)

Q14. Overflow - Signed and Unsigned

Unsigned Overflow:

- Occurs when carry out from MSB on addition
- CF = 1 indicates unsigned overflow
- Example: FFFFH + 0001H = 10000H (too big for 16 bits) => CF=1, OF=0

Signed Overflow:

- Occurs when result has a different sign than expected
- OF = 1 indicates signed overflow
- Example: 7FFFH + 7FFFH = FFFEH (two positives give negative) => OF=1, CF=0

Rules:

- Adding same sign numbers CAN overflow (signed)
- Adding different sign numbers CANNOT cause signed overflow
- Processor sets CF for unsigned overflow, OF for signed overflow

(Source: Chapter 5 - Section 5.2 PDF)

CHAPTER 6: ASSEMBLY LANGUAGE

Q15. Assembly Language Syntax - Fields of a Statement

An assembly language statement has up to 4 fields:

```
name     operation     operand(s)     comment
```

Field	Description	Example
Name	Instruction label, procedure name, or variable name	START: or MAIN
Operation	Opcode (for instruction) or Pseudo-op (for directive)	MOV, ADD, PROC, DW
Operand(s)	Data to be acted on. Can be 0, 1, or 2 operands	AX, WORD1, 5
Comment	Optional. Starts with semicolon (;)	; initialize counter

Example Instruction:

```
START:   MOV CX, 5   ; initialize the counter
```

Name=START:, Operation=MOV, Operands=CX and 5, Comment=;initialize the counter

Assembly Language Rules:

- Not case sensitive (but uppercase is preferred)
- Fields separated by at least one blank or tab
- Names: 1-31 characters, must start with a letter, not a digit

(Source: Chapter 4 - Assembly Language Syntax PDF)

Q16. Assembler Directives and Pseudo-Opcodes

Assembler Directive:

An assembler directive is a statement that instructs the assembler to perform some specific task (NOT converted to machine code). It allocates memory, defines segments, etc.

Pseudo-Opcode (Pseudo-op):

A pseudo-op is a code in the operation field of a directive. It tells the assembler what to do but is NOT translated into machine code.

Data Defining Pseudo-ops:

Pseudo-op	Stands For	Size	Example
DB	Define Byte	1 byte (8-bit)	ALPHA DB 4

Pseudo-op	Stands For	Size	Example
DW	Define Word	2 bytes (16-bit)	WORD1 DW 1000
DD	Define Doubleword	4 bytes (32-bit)	NUM DD 12345678H
DQ	Define Quadword	8 bytes (64-bit)	BIG DQ 0
DT	Define Ten Bytes	10 bytes	TEN DT 0
EQU	Equates (constant)	No memory	LF EQU 0AH

Memory Model Directives:

Directive	Description
.MODEL SMALL	Code in 1 segment (64KB), Data in 1 segment (64KB)
.MODEL MEDIUM	Code in multiple segments, Data in 1 segment
.MODEL COMPACT	Code in 1 segment, Data in multiple segments
.MODEL LARGE	Code in multiple segments, Data in multiple segments (no array > 64KB)
.MODEL HUGE	Code in multiple segments, Data in multiple segments (arrays > 64KB allowed)
.STACK size	Defines stack segment (e.g., .STACK 100H = 256 bytes)
.DATA	Defines data segment
.CODE	Defines code segment
PROC / ENDP	Defines start and end of a procedure
END	Marks end of the program

EQU Example:

```
LF EQU 0AH      ; assigns name LF to 0AH
MOV DL, LF      ; same as MOV DL, 0AH
```

Note: No memory is allocated for EQU names!

(Source: Chapter 4 - PDF Sections 4.1, 4.2, 4.3, 4.4, 4.7)

Q17. Program Structure - Putting it Together

```
.MODEL SMALL
.STACK 100H
.DATA
    ; data definitions go here
    A   DW   2
    B   DW   5
    SUM DW ?
.CODE
MAIN PROC
```

```
    MOV AX, @DATA    ; initialize DS
    MOV DS, AX
    ; instructions go here
    MOV AX, A
    ADD AX, B
    MOV SUM, AX
    MOV AX, 4C00H   ; exit to DOS
    INT 21H
MAIN ENDP
END MAIN
```

Why MOV AX, @DATA / MOV DS, AX is needed:

When a program loads, DOS places the PSP (Program Segment Prefix - 256 bytes) segment number in DS. So DS does not point to our data segment. We must manually initialize DS to point to our data.

(Source: Chapter 4 - PDF Section 4.7 + Class Notes)

Q18. Program Data - Numbers and Variables

Number Formats:

Number	Type	Rule
11011	Decimal	Default, no suffix needed
11011B	Binary	Must end with B or b
-21843D	Decimal	Optional D or d suffix
1B4DH	Hexadecimal	Must start with decimal digit AND end with H or h
FFFFH	ILLEGAL	Does NOT start with decimal digit
0FFFFH	Hexadecimal	CORRECT - starts with 0
1,234	ILLEGAL	Contains comma (non-digit character)

Variable Definitions:

```
ALPHA DB 4          ; byte variable, value = 4
BYT   DB ?          ; uninitialized byte variable
WRD   DW -2         ; word variable, value = -2
WRD   DW ?          ; uninitialized word variable
B_ARRAY DB 10H, 20H, 30H   ; byte array
W_ARRAY DW 1000, 40, 29887 ; word array
LETERS DB 'ABC'     ; same as DB 41H, 42H, 43H
MSG DB 'HELLO', 0AH, 0DH, '$'  ; string with control chars
```

(Source: Chapter 4 - PDF Sections 4.2, 4.3)

CHAPTER 7: BASIC INSTRUCTIONS

Q19. MOV and XCHG Instructions

MOV Instruction:

MOV transfers data between registers, register and memory, or moves a constant into register/memory.

```
Syntax: MOV destination, source

MOV AX, WORD1     ; memory to register
MOV AX, BX        ; register to register
MOV AH, 'A'       ; constant (41H) to register
MOV WORD1, AX     ; register to memory
```

XCHG Instruction:

XCHG exchanges (swaps) contents of two registers, or register and memory.

```
Syntax: XCHG destination, source

XCHG AX, WORD1    ; AX=0009H,WORD1=0008H => AX=0008H,WORD1=0009H
```

Restrictions on MOV and XCHG:

- CANNOT move between two memory locations directly: MOV WORD1, WORD2 is ILLEGAL
- CANNOT use constant as destination: MOV 5, AX is ILLEGAL
- CANNOT MOV between two segment registers directly
- Operands must be of same type (both byte or both word)

Workaround for memory-to-memory:

```
MOV AX, WORD1     ; load first to register
MOV WORD1, WORD2  ; ILLEGAL directly
MOV WORD2, AX     ; store from register
```

(Source: Chapter 4 - PDF Section 4.5.1)

Q20. ADD, SUB, INC, DEC, NEG Instructions

ADD and SUB Instructions:

```
Syntax:
ADD destination, source   ; destination = destination + source
SUB destination, source   ; destination = destination - source

Example:
ADD WORD1, AX   ; WORD1 = WORD1 + AX
SUB AX, DX      ; AX = AX - DX
```

INC and DEC Instructions:

```
Syntax:
INC destination   ; destination = destination + 1
DEC destination   ; destination = destination - 1
```

```
Example:
INC WORD1   ; WORD1: 0002H => 0003H
DEC WORD1   ; WORD1: FFFE H => FFFDH
```

NEG Instruction:

```
Syntax:
NEG destination   ; replaces with its 2's complement (negate)

Example:
NEG BX   ; BX: 0003H => FFFDH (which is -3 in signed)
```

Restrictions on ADD/SUB:

- CANNOT add/subtract two memory locations directly: ADD BYTE1, BYTE2 is ILLEGAL
- Workaround: MOV AL, BYTE2 then ADD BYTE1, AL

Type Agreement:

- Both operands must be same type (both byte or both word)
- MOV AX, BYTE1 is NOT allowed (word register + byte variable)

(Source: Chapter 4 - PDF Section 4.5.2)

CHAPTER 8: INPUT/OUTPUT AND INTERRUPTS

Q21. DOS and BIOS Routines - Differences and Examples

Feature	BIOS Routines	DOS Routines
Storage	Stored in ROM (Read-Only Memory)	Stored in DOS operating system
Level	Low-level, directly interacts with I/O ports	High-level, uses BIOS routines internally
Function	Basic screen operations (cursor, scroll)	Complex tasks (print string, file operations)
Interrupt	INT 10H (video), INT 16H (keyboard)	INT 21H (most common)
Portability	Hardware dependent	More portable across PC models
Speed	Faster (direct hardware access)	Slightly slower (goes through BIOS)
Example	INT 10H for screen scroll	INT 21H for character input/output

(Source: Chapter 4 - PDF Section 4.8)

Q22. INT 21H - DOS Interrupt with Examples

INT 21H is the most commonly used DOS interrupt. A function number is placed in AH register before calling INT 21H.

Function (AH=)	Task	Input	Output
01H	Single-key Input	AH = 1	AL = ASCII code of key pressed (0 if non-character key)
02H	Single Character Output	AH = 2, DL = ASCII code to display	Displays character on screen
09H	String Output	AH = 9, DX = offset address of string (ending with '$')	Displays string on screen
4CH	Exit to DOS	AH = 4CH	Terminates the program

Example - Single Character Input:

```
MOV AH, 1
INT 21H     ; AL = ASCII code of key pressed
```

Example - Single Character Output:

```
MOV AH, 2
MOV DL, 'A'   ; DL = 41H (ASCII of 'A')
INT 21H        ; displays 'A' on screen
```

Example - String Output:

```
.DATA
MSG DB 'HELLO$'   ; string MUST end with '$'
.CODE
MOV AX, @DATA
MOV DS, AX
MOV AH, 9
LEA DX, MSG    ; DX = offset address of MSG
INT 21H        ; displays 'HELLO' on screen
```

Control Characters for INT 21H Function 2:

ASCII (Hex)	Symbol	Function
07H	BEL	Beep (sounds a tone)
08H	BS	Backspace
09H	HT	Tab
0AH	LF	Line Feed (new line)
0DH	CR	Carriage Return (go to start of line)

Newline Implementation:

```
MOV AH, 2
MOV DL, 0AH   ; Line Feed
INT 21H
MOV DL, 0DH   ; Carriage Return
INT 21H        ; together = new line
```

LEA Instruction:

LEA = Load Effective Address. Loads the OFFSET address of a variable into a register.

```
LEA DX, MSG   ; DX = offset address of variable MSG
              ; needed before INT 21H function 9
```

(Source: Chapter 4 - PDF Sections 4.8, 4.9, 4.11)

Q23. Program Segment Prefix (PSP) - What is it?

When a program is loaded in memory, DOS places a 256-byte (100H) block called the PSP (Program Segment Prefix) before the program.

- PSP contains information about the program
- DOS places PSP segment number in DS and ES before program execution
- Therefore DS does NOT initially point to our program's data segment
- We must fix this at the start of every program that has a data segment:

```
MOV AX, @DATA   ; get data segment number
MOV DS, AX      ; put it in DS
```

@DATA = assembler symbol for the data segment defined by .DATA directive

(Source: Chapter 4 - PDF Section 4.11 + Class Notes)

CHAPTER 9: COMPLETE SAMPLE PROGRAMS

Q24. Complete Assembly Language Programs

Program 1: Add Two Numbers (From PDF)

```
TITLE PGM1_1: SAMPLE PROGRAM
.MODEL SMALL
.STACK 100H
.DATA
    A   DW  2
    B   DW  5
    SUM DW  ?
.CODE
MAIN PROC
    MOV AX, @DATA    ; initialize DS
    MOV DS, AX
    MOV AX, A        ; load A into AX
    ADD AX, B        ; AX = A + B
    MOV SUM, AX      ; store result
    MOV AX, 4C00H   ; exit to DOS
    INT 21H
MAIN ENDP
END MAIN
```

Program 2: Echo Program - Read char, display on next line

```
TITLE PGM4_1: ECHO PROGRAM
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV AH, 2
    MOV DL, '?'       ; display prompt '?'
    INT 21H
    MOV AH, 1
    INT 21H           ; read character into AL
    MOV BL, AL        ; save character in BL
    MOV AH, 2
    MOV DL, 0DH       ; carriage return
    INT 21H
    MOV DL, 0AH       ; line feed
    INT 21H
    MOV DL, BL        ; restore character
    INT 21H           ; display character
    MOV AH, 4CH
    INT 21H           ; exit
MAIN ENDP
END MAIN
```

Program 3: Print String

```
TITLE PGM4_2: PRINT STRING PROGRAM
.MODEL SMALL
.STACK 100H
.DATA
    MSG DB 'HELLO$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    LEA DX, MSG       ; DX = offset of MSG
    MOV AH, 9
    INT 21H           ; print the string
    MOV AH, 4CH
    INT 21H           ; exit
MAIN ENDP
END MAIN
```

(Source: Chapter 1 & 4 - PDF Sections 1.5, 4.9, 4.11)

QUICK REFERENCE SUMMARY TABLE

8086 Key Facts at a Glance

Topic	Key Fact
Processor Type	16-bit microprocessor
Data Bus	16-bit (can do 8 or 16-bit operations)
Address Bus	20-bit
Memory Capacity	2^20 = 1 MB = 1,048,576 bytes
I/O Ports	2^16 = 64K ports
Instruction Length	1 to 6 bytes
Instruction Queue	6 bytes
Total Registers	14 sixteen-bit registers
Flags	9 active flags (6 status + 3 control)
Two Main Units	Execution Unit (EU) + Bus Interface Unit (BIU)
EU Registers	AX, BX, CX, DX, SI, DI, BP, SP + FLAGS
BIU Registers	CS, DS, SS, ES, IP + Instruction Queue
Physical Address Formula	Segment x 10H + Offset
Segment Size	64 KB = 65,536 bytes
Paragraph	16 bytes (10H)
Data Bus (8085)	8-bit (8086 is 2x wider)
Memory Ratio 8086/8085	16 times more memory access
ALE Pin	Address Latch Enable - used for demultiplexing
Program Exit	MOV AX, 4C00H + INT 21H
Initialize DS	MOV AX, @DATA + MOV DS, AX

Most Important Exam Formulas

```
Physical Address = Segment x 10H + Offset

Memory Capacity = 2^N  (N = address bus width)
8086: 2^20 = 1 MB
8085: 2^16 = 64 KB

N-bit processor => N = data bus width

Segment:Offset => Physical Address:
  Example: A4FB:4872
  A4FB0H + 4872H = A9822H

Find Segment from Physical and Offset:
```

```
Segment = (Physical - Offset) / 10H
```

END OF DOCUMENT - All answers based on Course PDF (Chapters 1-5) & Class Notes